Project2 Statistics 01204314

Author: Kongphop Tirattanaprakom 6610505250

Part 1 (Dataset C & B)

EDA

My decision is not to add any additional changes. As from the previous EDA is quite reasonable for what to do with the categorical and numberical data, the EDA part will use the same procedure as before in the project 1.

Clustering

Using KMeans clustering as a base model for separate mini regression models. By selected k of 4 produce these performance evaluation:

- Silhouette Index: 0.1370
- Davies-Bouldin Index: 2.3052
- Dunn Index: 0.2735
- CH Index: 121.3078

By simulates with elbow method and silhouette score, number of cluster selected is 4, Even if the silhouette score of 5 is better when observing in the silhouette chart. Because when separating the training data, it doesn't separate well as one cluster contains small amount of training data which selected by the cluster itself.

Therefore 4 is the number that the author select instead and it does perform the separation pretty even.

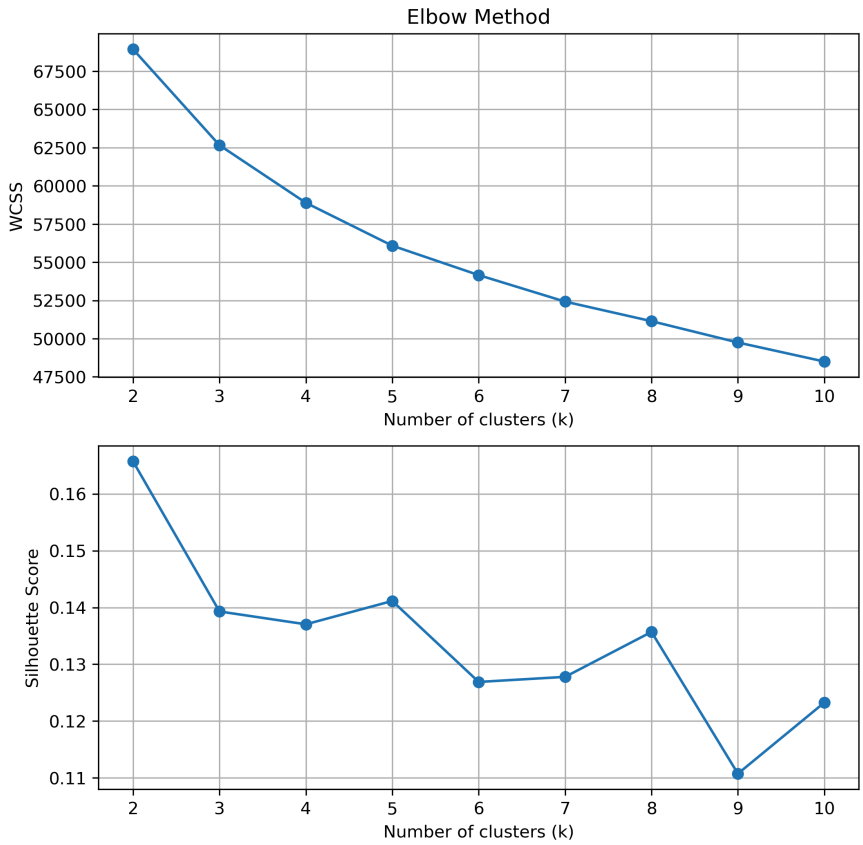

Figure 1: Simulation of WCSS (elbow method) and silhouette score

From the author perspective of these variables which are “Rural population (% of total population)”, “Agricultural land (% of land area)”, “Crop production index (2014-2016 = 100)”, “Merchandise trade (% of GDP)” and “Access to electricity (% of population)”, would characterized countries or economic structures inside each cluster.

Figure 2: Radar graph for selected variables

The cluster 1 is the best to explain the agricultural careers cluster. As the higher the agricultural land and rural population, the crop production increases in the rural perspective. On the other hand, cluster 0 which represents urban perspective with a higher of access to electricity, they seem to keep their GDP with merchandise trade. The other clusters show in between of these 2 extreme rural and urban clusters. As the increasing of the rural population, agricultural land and crop production index, it would decrease the merchandise trade and access to electricity

Features were selected using 2 layers of feature selection. The first layer is to remove highly correlated variables and the second layer is a recursive feature elimination was used to drop the weak features.

For the regression model, with the respect of the small data in each training dataset for each cluster, the author decides to use a simple model which is Ridge regression or L2 regression. Ridge by itself shrinks the coefficients of less important features against outlier and high variance. The hyperparameter for Ridge is a penalty strength alpha.

Both feature subset selection and hyperparameter alpha were selected from the sklearn Pipeline and GridSearchCV. With the help from KFold algorithm, it can evaluated dozens of combinations and objectively selected the parameters that minimized the Root Mean Squared Error (RMSE)

Based on the experimental results, the optimal number of features ($\mathrm{n_features}$), the ideal regularization strength (α), training root mean square error (RMSE) and top three most important features for each cluster's Ridge regression are listed below:

Cluster 0
- Number of features: 40
- Alpha: 0.1
- RMSE: 1,901.29
- Top three most important features: 'GDP (current US$)', 'Population, total', 'Merchandise trade (% of GDP)'

Cluster 1
- Number of features: 50
- Alpha: 0.01
- RMSE: 669.53
- Top three most important features: 'Rural population', 'GDP (current US$)', 'Region_Middle East'

Cluster 2
- Number of features: 50
- Alpha: 0.1
- RMSE: 2,490.63
- Top three most important features: 'Population ages 65 and above (% of total population)', 'GDP (current US$)', 'Rural population'

Cluster 3
- Number of features: 50
- Alpha: 0.01
- RMSE: 4,811.51
- Top three most important features: 'GNI, PPP (current international $)', 'Population, total', 'Total reserves (includes gold, current US$)'

Overall Performance Evaluation

Overall Piecewise RMSE, when using test dataset by separate each into the nearest cluster and then perform a regression prediction, is 4,244.53 while overall R-squared is 0.9585.

Observes each cluster's RMSE from the test dataset:
- Cluster 0: test_amount = 42 and RMSE = 2,583.75
- Cluster 1: test_amount = 61 and RMSE = 1,460.99
- Cluster 2: test_amount = 48 and RMSE = 3,405.87
- Cluster 3: test_amount = 65 and RMSE = 6,707.14

From the RMSE and top three most important features, the cluster 1 perform with least RMSE compared to the others and cluster 3 perform the worst. For cluster 0, the most important features are likely to be an urban cluster. And for cluster 1 which represent rural, the most important feature is 'Rural population'. The intersect features between clusters, which is in the top three most important features, are 'GDP (current US$)', 'Population, total' and 'Rural population'.

Part 2

Part 2.1 (Text Processing)

Word2Vec with RandomForestClassifier

The Word2Vec model converts words into dense vectors of size 50 and then represent the entire article by taking the average of all the word vectors in that text. Working with a RandomForestClassifier that uses 100 decision trees and criterion of gini.

Performance:

Confusion Matrix

$$\begin{bmatrix} 32 & 1 & 10 \\ 1 & 53 & 0 \\ 2 & 1 & 46 \end{bmatrix}$$

Accuracy: 0.8973
Precision: 0.8998
Recall: 0.8881
F1 score: 0.8897

TF-IDF with LogisticRegression

The TF-IDF (Term Frequency-Inverse Document Frequency) model creates a high-dimensional vector representing the frequency of words in an article, weighted by how rare and uniquely identifying they are across the dataset. Working with a LogisticRegression that take maximum of 1000 iterations for the solver to converge.

Performance:

Confusion Matrix

$$\begin{bmatrix} 42 & 0 & 1 \\ 0 & 54 & 0 \\ 0 & 0 & 49 \end{bmatrix}$$

Accuracy: 0.9932
Precision: 0.9933
Recall: 0.9922
F1 score: 0.9927

Compare the performance

TF-IDF with LogisticRegression significantly outperforms Word2Vec with RandomForestClassifier across all metrics.

The reasons, why it does outperform in a huge gap, is that while Word2Vec is excellent at capturing semantic meaning, it is simply averages all the word vectors together to represent a long article. Author opinion is that after averaging those words into a single vector, so it might washed out the highly discriminative keywords. On the other hand, TF-IDF model, which specific vocabulary is the strongest indicator of the category, achieving near perfect classification.

Secondly, by using a simple LogisticRegression on a sparse data, it can easily draws a clear linear boundaries. As for RandomForestRegression on a Word2Vec output vectors, It may catch keywords, which separate the categories, with its overfitting trees. But in the end, if the data is too sparse, then it cannot overcome this 50-dimensional dense space problem.

Identify top 10 most frequent words

1. Label 0

said: 430	film: 408	best: 358	year: 232	music: 230
also: 205	one: 198	new: 178	show: 161	last: 159

Label 0 would inferred -> Entertainment show or music.

2. Label 1

said: 1298	would: 637	labour: 438	government: 426	people: 383
party: 366	election: 328	blair: 316	new: 278	could: 252

Label 1 would inferred -> Politics and economics.

3. Label 2

said: 498	year: 290	first: 259	world: 238	game: 235
time: 228	england: 227	win: 226	two: 222	last: 220

Label 2 would inferred -> Gaming and world tournament.

Identify top 10 most similar words

1. Similar words to “technology”: keep, written, north, become, return, judge, area, part, led, something
2. Similar words to “finance”: bill, stree, move, forme, earlie, choic, may, forwar, home, new

Part 2.2 (Time Series)

Firstly observes the data with null percent, found two columns with Nan value, which after list all the unique value it does contain only garbage value, So I decides to drop those columns which are ‘SNOW’ represents snowfall and ‘SNWD’ represents snow depth.

SARIMA

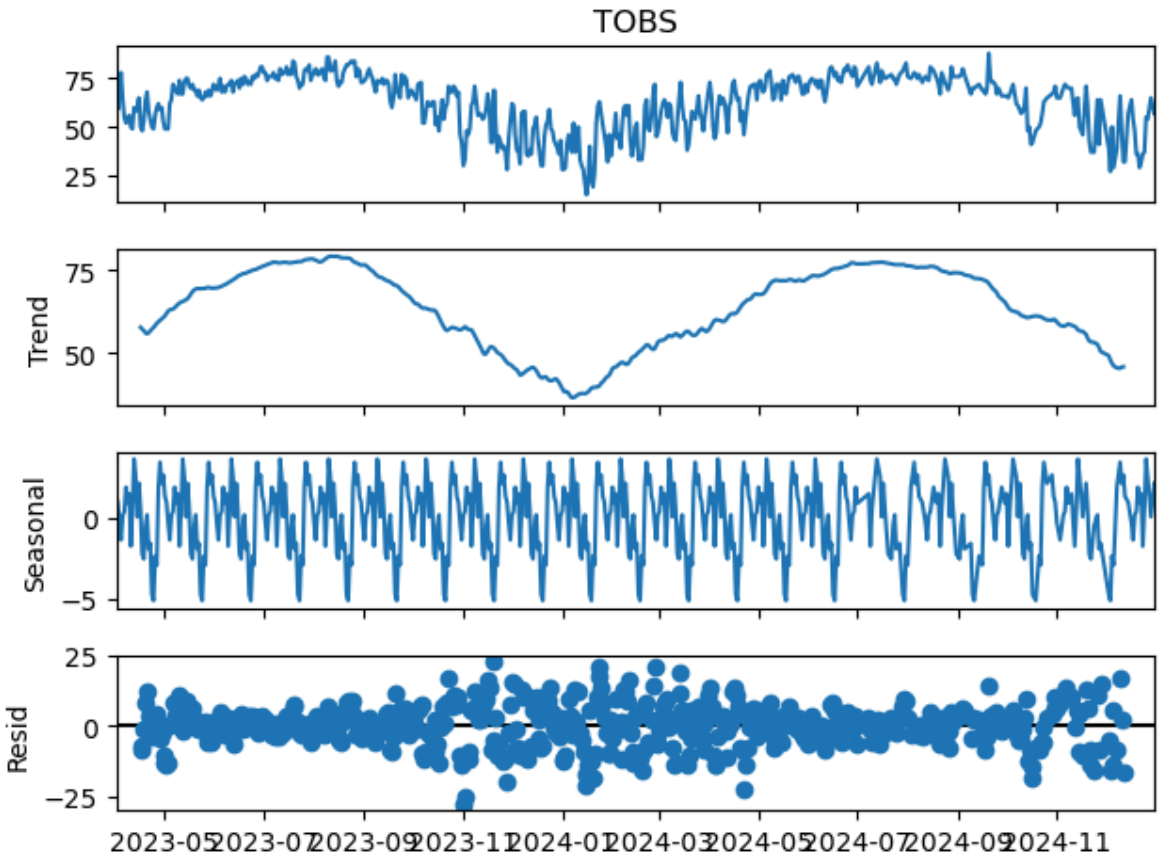

Figure 3: Seasonal decompose of temperature at the time of observation (TOBS)

Plotting the seasonal decompose with additive model and period of 30, which selected s is 30. With the test adfuller of differencing d, the data is stationary after the first difference, therefore the selected d is 1. Seasonal difference is the difference with shifted selected s. The data is not stationary, but its first difference does. Therefore selected D is 1.

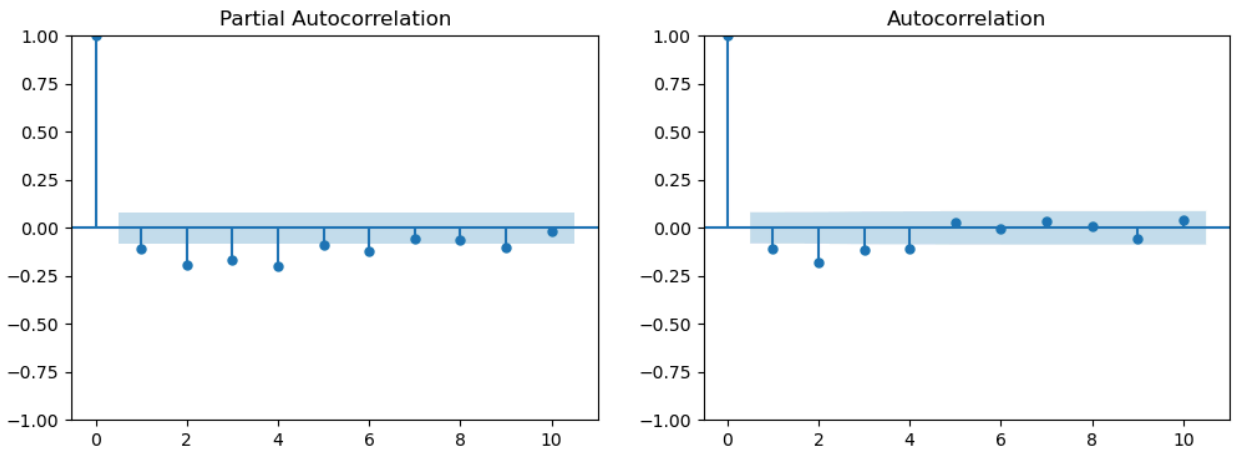

Figure 4: Identify p and q from PACF and ACF for timeseries

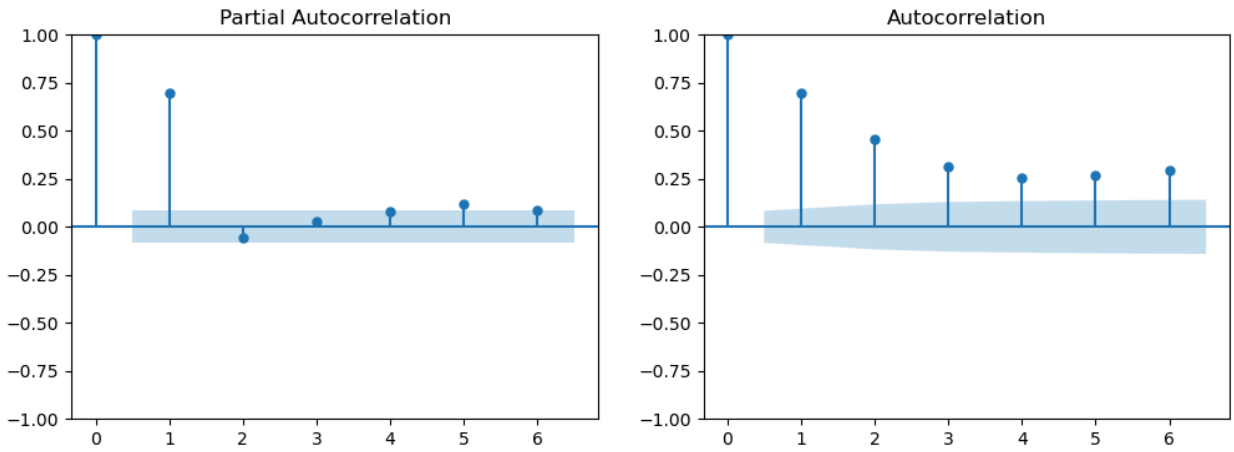

Figure 5: Identify P and Q from PACF and ACF for seasonal timeseries

From the autocorrelation function (ACF) and partial autocorrelation function (PACF), the selected p, q from the graph observation is 1, 1 and selected P, Q is 1, 0.

Selected [(p, d, q), (P, D, Q, s)] is [(1, 1, 0), (1, 1, 0, 30)] results in

- AIC: 4019.8229
- Prob(Q): 0.8614
- Prob(H): 0.7678

For AIC, the lower the value, the better model. Prob(Q) > 0.05 checks if the errors of the model made are correlated with each other. It fails to reject the null hypothesis, which means the errors are random and independent. Prob(H) > 0.05 checks if the variance (the spread of size of errors) remains constant over time, which means the errors have a constant variance over time.

The results evaluation
- RMSE: 17.6862
- MAE: 14.7087

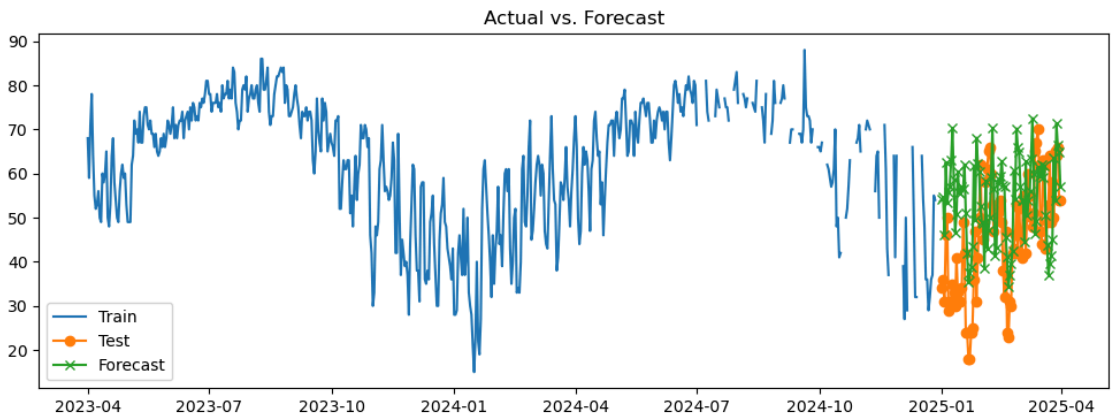

Figure 6: SARIMA model prediction

LSTM with lags

Creates a lags of temperature at the time of observation, with 12 lags shifted and its original TOBS value. Then scales with standard scaler and creates sequences plus separates into train and test data.

Train the LSTM model with hyperbolic tangent (Tanh) activation function with window size of 14 and adaptive moment estimation (Adam) optimizer with a loss of mean square error (MSE) for 100 epochs.

The results evaluation

- RMSE: 8.9297
- MAE: 6.6329

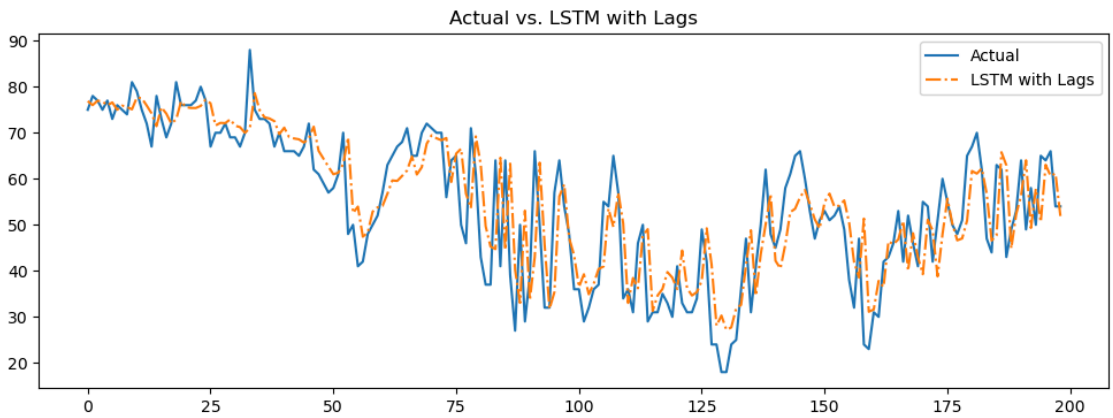

Figure 7: LSTM with lags prediction

RNN with features

From the preprocessing, the dropped columns were ‘SNOW’ - snowfall and ‘SNWD’ - snow depth. The remaining features are ‘TMAX’ - maximum temperature, ‘TMIN’ - minimum temperature, and ‘PRCP’ - precipitation which will be all selected to train the RNN model.

Train RNN model with rectified linear unit (ReLU) activation function with Adam optimizer and MSE loss for 20 epochs.

The results evaluation

- RMSE: 5.0276
- MAE: 4.1591

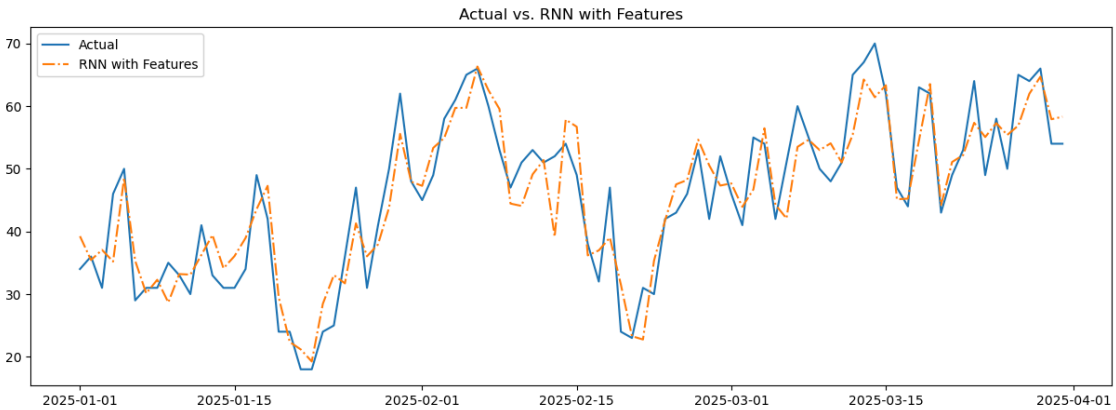

Figure 8: RNN with selected features prediction

Performance comparison

The RNN with all remaining features perform best of all three models and the SARIMA performs worst. As the data is not possible for complete the 2 years seasonal timeseries for SARIMA, therefore its perform based on period of a month instead. It cannot predicts the whole year of temperature at the time of observation which in the author perspective, it does not change quite much by each year. Unlike lags which use its past to predict present or features training which temperature likely to depend mostly on those selected features and its past value.

Part 2.3 (Image Processing)

VGG16 pretrained model

The model uses the VGG16 convolutional base, which contains 14,714,688 parameters dedicated to extracting image features. Custom layers, which are average pooling layer, batch normalize layer, dense layer with ReLU activation reduce to the output size of classes with softmax activation function to determine the answer, contain 67,204 parameters in total.

VGG16 performance

- Test loss: 0.6116
- Test accuracy: 0.7875

ResNet50 pretrained model

The model utilizes the ResNet50 base, which provides a total of 23,587,712 parameters for feature extraction. The custom layers, which are the same as VGG16 chosen layers, contain 266,884 parameters.

ResNet50 performance

- Test loss: 0.3870
- Test accuracy: 0.8625

Performance comparison and details

Because ResNet50 utilizing significantly more depth and complex, the performance is also significantly higher accuracy compared to VGG16 transfer learning. For author opinion, because the data is about image processing the more complex with higher amount of parameters might grep a better separation between each class.

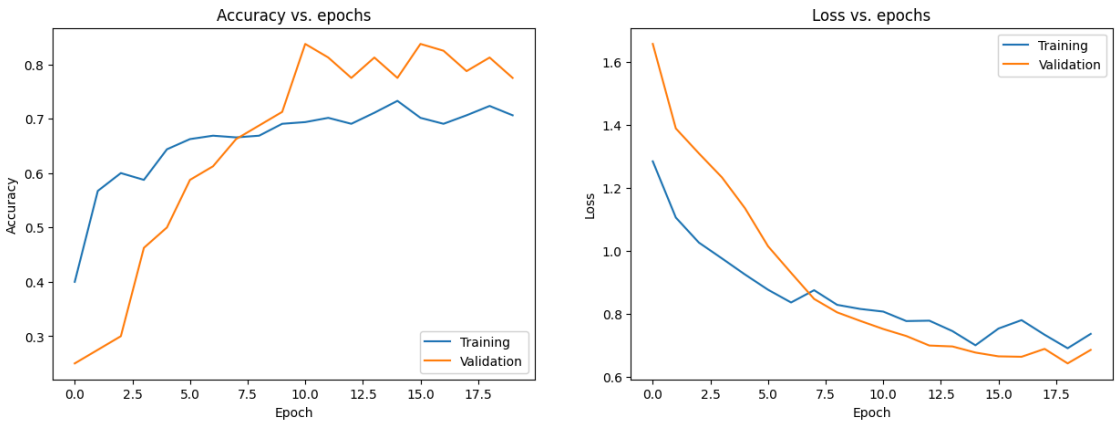

Figure 9: VGG16 based performance during training and validation phases

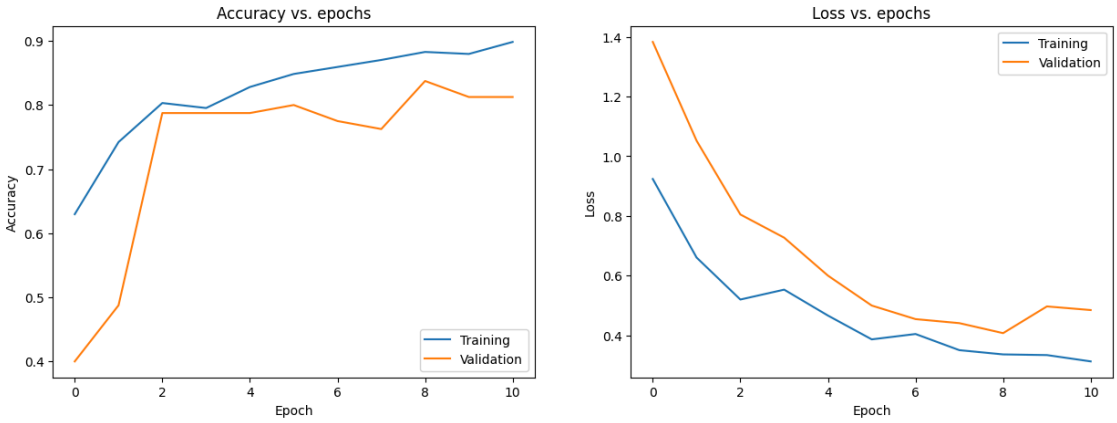

Figure 10: ResNet50 based performance during training and validation phases

Model VGG16 based shows slow and steady convergence. Also its validating data outperform the training data on accuracy when after it trains for about 8 epochs. ResNet50 based on the other hand, it learns rapidly as the training accuracy increase and maintains lower training and validation loss throughout the process.

The ResNet50 based model was highly efficient, as it learned the apparel pattern almost twice as fast as VGG16 based. This convergence is evident in the loss metrics. By the end of training, the ResNet50 model's loss had dropped to approximately 0.4, whereas the VGG16 model's loss remained around 0.8.

VGG16-based Confusion Matrix

$$\begin{bmatrix} 15 & 1 & 2 & 2 \\ 1 & 18 & 0 & 1 \\ 1 & 3 & 16 & 0 \\ 4 & 2 & 0 & 14 \end{bmatrix}$$

From VGG16-based confusion matrix,

True dress: Mistaken for pants (1), shirt (2), shoes (2)

True pants: Mistaken for dress (1), shoes (1)

True shirt: Mistaken for dress (1), pants (3)

True shoes: Mistaken for dress (4), pants (2)

ResNet50-based Confusion Matrix

$$\begin{bmatrix} 15 & 1 & 1 & 3 \\ 0 & 20 & 0 & 0 \\ 1 & 1 & 18 & 0 \\ 2 & 2 & 0 & 16 \end{bmatrix}$$

From ResNet50-based confusion matrix,

True dress: Mistaken for pants (1), shirt (1), shoes (3)

True pants: 0 error

True shirt: Mistaken for dress (1), pants (1)

True shoes: Mistaken for dress (2), pants (2)

The similarities of both two models are they share a very specific weakness which is confusing dress and shoes. As they both predicted actual dresses as shoes. And the difference is that the ResNet50-based outperforms VGG16-based with a flawless prediction for pants (100% recall performance). Also VGG16-based heavily confused shirts for pants whereas ResNet50-based mostly solves this issue.